EDGE LAB ASSESSMENT

SURAJ K M

25MML0047

1. DATA Lab-3 utilized the same large-scale log dataset: • logs_dataset.csv containing 721,547 log entries • Includes timestamps, unique identifiers, and IP addresses • Suitable for advanced analytical tasks such as: • Pattern mining • Anomaly detection • Clustering • Time-series feature generation
2. TASK The objective was to perform deeper analysis of log patterns by: • Transforming timestamps into numerical and time-based features • Encoding and processing IP address information • Preparing the dataset for machine learning applications such as: • Clustering • Anomaly detection • Trend and behavior analysis
3. MODEL Based on the notebook implementation, Lab-3 involved: • Clustering algorithms (K-Means, DBSCAN) • Anomaly detection techniques • Statistical analysis approaches • Time-series pattern extraction The main focus was on applying machine learning and analytical methods to real-world log data.
4. LOSS / CHALLENGES • Feature sparsity caused by categorical IP address patterns • Difficulties in parsing and handling timestamp formats • Increased memory usage due to the large dataset size • Presence of noise in synthetically generated IP

address values

[35]:
```
import pandas as pd
import numpy as np
```

[33]:
```
df=pd.read_csv("logs_dataset.csv")
```

[34]:
```
df.head()
```

[34]:

	@timestamp	_id	ip_address
0	July 8th 2019, 14:43:03.000	XswJ0msBoTGddM7vxMDB	10.1.1.285
1	July 8th 2019, 14:43:01.000	dKQJ0msB7mP0GwVzvJjz	10.1.2.389
2	July 8th 2019, 14:42:59.000	CcwJ0msBoTGddM7vtb8y	10.1.1.415
3	July 8th 2019, 14:42:57.000	bKQJ0msB7mP0GwVzrZdT	10.1.1.79
4	July 8th 2019, 14:42:55.000	L6QJ0msB7mP0GwVzpZel	10.1.1.60

[36]:
```
len(df)
```

[36]: 721547

```
import pandas as pd
import numpy as np
df=pd.read_csv("/content/logs_dataset.csv")
```

```
df['@timestamp'] = (
    df['@timestamp']
    .str.replace(r'(\d+)(st|nd|rd|th)', r'\1', regex=True)
)
df['@timestamp'] = pd.to_datetime(
    df['@timestamp'],
    format='%B %d %Y, %H:%M:%S.%f'
)
```

```
df.sort_values(['ip_address', '@timestamp'], inplace=True)
```

```
df['shift_time'] = df.groupby(['ip_address'])['@timestamp'].shift(1)
```

```
df.head()
```

	@timestamp	_id	ip_address	shift_time
102113	2019-07-05 15:08:55	xayuwmsBoTGddM7vX3ZD	10.1.1.1	NaT
101735	2019-07-05 15:24:39	ma28wmsBoTGddM7vxo3E	10.1.1.1	2019-07-05 15:08:55
101381	2019-07-05 15:39:25	xIXKwmsB7mP0GwVzS_HE	10.1.1.1	2019-07-05 15:24:39
101191	2019-07-05 15:47:19	vK_RwmsBoTGddM7vhxpD	10.1.1.1	2019-07-05 15:39:25
100621	2019-07-05 16:11:05	mLDnwmsBoTGddM7vSa2T	10.1.1.1	2019-07-05 15:47:19

```
df['time_diff'] = (df['@timestamp']-df['shift_time']).dt.seconds//60
```

```
df['date'] = df['@timestamp'].dt.date
```

```
df['weekday'] = df['@timestamp'].dt.weekday
```

```
df['hour'] = df['@timestamp'].dt.hour
```

```
df['is_weekend'] = ((df['weekday'] == 5) | (df['weekday'] == 6)).astype(int)
```

```
df.head()
```

	@timestamp	_id	ip_address	shift_time	time_diff	date	weekday	hour	is_weekend
102113	2019-07-05 15:08:55	xayuwmsBoTGddM7vX3ZD	10.1.1.1	NaT	NaN	2019-07-05	4	15	0
101735	2019-07-05 15:24:39	ma28wmsBoTGddM7vxo3E	10.1.1.1	2019-07-05 15:08:55	15.0	2019-07-05	4	15	0
101381	2019-07-05 15:39:25	xlXKwmsB7mP0GwVzS_HE	10.1.1.1	2019-07-05 15:24:39	14.0	2019-07-05	4	15	0
101191	2019-07-05 15:47:19	vK_RwmsBoTGddM7vhxpD	10.1.1.1	2019-07-05 15:39:25	7.0	2019-07-05	4	15	0
100621	2019-07-05 16:11:05	mLDnwmsBoTGddM7vSa2T	10.1.1.1	2019-07-05 15:47:19	23.0	2019-07-05	4	16	0

```
df['hour_bucket'] = df['hour']//4
# This integer division by 4, so we can analyse 4-hour buckets instead of every single hour
```

```
ip_addr ='ip_address'
```

```
ip_counts = df.groupby(ip_addr)['@timestamp'].count().reset_index()
```

```
ip_counts.head()
```

	ip_address	@timestamp
0	10.1.1.1	205
1	10.1.1.100	432
2	10.1.1.101	196
3	10.1.1.106	199
4	10.1.1.109	221

```
ip_counts = ip_counts.rename(columns={'@timestamp':'total_count'})
```

```
daily_counts = df.groupby([ip_addr,'date'])['@timestamp'].count().reset_index()
```

```
daily_counts = daily_counts.rename(columns={'@timestamp':'daily_counts'})
```

```
daily_counts_avg = daily_counts.groupby(ip_addr).daily_counts.median().reset_index()
```

```
daily_counts_avg.head(5)
```

	ip_address	daily_counts
0	10.1.1.1	56.5
1	10.1.1.100	115.5
2	10.1.1.101	44.5
3	10.1.1.106	50.0
4	10.1.1.109	58.5

```
weekend_counts = df.groupby([ip_addr, 'is_weekend'])['@timestamp'].count().reset_index()
```

```
weekend_counts
```

	ip_address	is_weekend	@timestamp
0	10.1.1.1	0	73
1	10.1.1.1	1	132
2	10.1.1.100	0	139
3	10.1.1.100	1	293
4	10.1.1.101	0	61
...	...	...	...
767	10.1.2.90	1	270
768	10.1.2.95	0	118
769	10.1.2.95	1	268
770	10.1.2.99	0	61
771	10.1.2.99	1	136

772 rows × 3 columns